TRACK 2 DISPATCH / TEAM LEARNING SERIES

DOCUMENT 2 OF 2

Track 2 Dispatch explained simply

The conversation explanation, prepared for your team

A standalone reading of the project-review explanation: what has been built, how the code works, what the results mean, and what the team should build next. Includes the worked example and presentation script.

Implementation snapshot: 14 September 2026
The allocation engine is built. Conversational AI and the dispatcher interface remain planned work.

Prepared for reading, presentation preparation, and team sharing

PDF edition: 15 September 2026

Code lines may wrap in print; use the Markdown source when copying commands.

Contents

Click a chapter title or use the PDF bookmarks to navigate.

1. What we have built so far 3

2. The business problem 3

3. What the assignment requires 3

4. How the components work together 4

5. Folders, packages, and supporting files 5

6. The data and the two clocks 6

7. The supplied simulator 7

8. The tools and a worked example 8

9. Objectives: what does “best” mean? 10

10. The allocator and decision record 12

11. Detecting unexpected queue growth 13

12. The heuristic and evaluation files 15

13. Tests and measured results 15

14. What to build next and the deadlines 16

15. A presentation explanation to practise 17

16. Source documents and scope 18

This document preserves the substantive explanation from our project-review conversation, arranged for reading and sharing. It describes the code reviewed on 14 September 2026. PDF prepared on 15 September 2026. Local chat links have been replaced with project-relative file references, and the workflow diagram has been redrawn for print.

The companion PDF, `project_learning_guide.pdf`, contains the longer file-by-file walkthrough, a beginner's Python glossary, explanations of all 30 chat requests, worked calculations, results, presentation questions, and where to edit when requirements change.

1. What we have built so far

I read the seven capstone PDFs, compared their versions, and reviewed the project's source code, data, tests, and documentation.

The most important thing to understand is that we have built the allocation decision engine. The conversational AI and user interface still need to be built. The current program makes decisions using explicit Python rules and calculations. It does not currently call ChatGPT or another language model.

2. The business problem

Your project helps a factory decide which outside workshop should receive a clothing order.

The factory, SweaterCo, sometimes receives more work than it can handle internally. It sends some batches to outside workshops.

A *batch* means a quantity of garments being sent together - for example, 340 sweaters.

There are eight workshops. Each has different characteristics:

- What garments it can make.
- How many pieces it can produce per day.
- How much work is already waiting.
- How long transport takes.
- Its price.
- Its chance of producing a batch requiring rework.
- Restrictions on whether it can accept the order.

The business problem is that **the cheapest workshop, the largest workshop, and the workshop that can finish soonest may be different places**.

Our intended system helps a dispatcher compare those choices and explain the decision. That is the manufacturing context described in the supplied `Factory_Primer.pdf`.

3. What the assignment requires

The assignment requires more than a program that chooses workshops.

The completed system must handle messages such as:

> *Send ORD-073 to FreshStart.*

It should look up the order, discover that it contains 400 hoodies, check FreshStart's 300-piece limit, and explain why that workshop cannot take it.

It should also handle:

> *Send the usual quantities.*

Here, it may need clarification.

And:

> *Which workshop handled this customer's order last October?*

Here, it must acknowledge that the required history is unavailable rather than inventing an answer.

The final system therefore needs two connected abilities:

Ability	What it means	Current status
Understand the dispatcher	Interpret messages, resolve order references, ask questions, recognise unavailable facts	Still to build
Evaluate an allocation	Check restrictions, calculate estimates, rank choices, record reasons	Implemented for structured inputs

The latest Track 2 description explicitly says that an allocation policy without a natural-language interface does not meet the core capabilities. Our current engine is an important foundation, but it is not yet the completed capstone. Source: `Documents_Capstone/Capstone_Project_Tracks_v3.pdf`.

4. How the components work together

Think of the current program as a dispatcher working inside a practice factory.

The simulator creates that practice world. Our allocator makes decisions within it.

1 Orders and workshop data

↓

2 Simulator supplies an order and current queues

↓

3 Monitor checks for unexpected queue growth

↓

4 Tools check legal workshops and calculate estimates

↓

5 Objective formula scores the choices

↓

6 Allocator selects a workshop and records its reasons

↓

7 Simulator processes the order and updates queues

↓

8 Repeat for the next batch

The simulator controls what actually happens in the experiment. The allocator estimates what will happen and chooses where to send work.

This separation lets us test whether our decisions are useful.

5. Folders, packages, and supporting files

Each folder has a specific job.

Folder	Simple meaning
`data/`	The information the program reads
`harness/`	The practice factory and its scoring rules
`src/`	Our decision-making code
`eval/`	Experiments and saved results
`docs/`	Written explanations
`tests/`	Automatic checks that the code behaves as intended

The supporting files matter too:

- `requirements.txt` lists packages to install.
- `.venv/` contains this project's separate Python environment.
- `.env.example` shows where a future API key setting belongs.
- `.gitignore` keeps local secrets, environments, and generated caches out of Git.

- `README.md` explains initial setup.
- `Factory_Primer.md` explains the business setting.
- `docs/tool_spec.md` documents what each tool does and how it can fail.

Packages such as `openai`, `streamlit`, and `matplotlib` are installed or listed in preparation for future work. **Their presence does not mean an AI assistant, interface, or charting feature has already been implemented.**

GitHub stores the project's source code and history. It does not automatically turn that code into a working website.

6. The data and the two clocks

The data consists of orders, workshop profiles, and messages.

`orders.csv` contains **120 orders totalling 93,500 pieces**.

It includes:

- An order ID.
- Customer and product.
- Category.
- Quantity.
- Order and due dates.
- Recorded progress and completion information.

There are two broad product categories:

- `TOPS`: sweaters, hoodies, vests, and similar garments.
- `ACCESSORIES`: scarves and beanies.

The simulator uses only the fields it needs for allocation: ID, category, pieces, dispatch date, and due date.

It replays **all 120 orders**, including orders marked complete in the historical data. It is performing a fresh experiment, not continuing the historical factory schedule.

`workshops.csv` contains these eight profile cards:

Workshop	Pieces / day	Transport days	Batch defect chance	Cost / piece	Main restriction
W1 QuickStitch	300	1	12%	1.50	TOPS only
W2 SteadyHands	170	2	3%	1.30	Both categories
W3 BudgetWorks	230	3	9%	0.80	TOPS only
W4 Little Loom	70	1	2%	1.60	ACCESSORIES only
W5 GiantWeave	400	4	6%	1.10	TOPS only
W6 Nimble Needle	130	1	5%	1.20	Both categories
W7 OldMill	200	2	10%	0.90	Suspended
W8 FreshStart	100	2	4%	1.00	TOPS, maximum 300 pieces per batch

Cost is in the simulator's price units; the supplied data does not establish a currency.

A workshop's daily capacity is different from a batch limit. SteadyHands can accept 340 pieces even though its capacity is 170 per day: it takes two days. FreshStart's 300-piece cap is a separate restriction that makes a 340-piece batch illegal.

`dispatch_requests.txt` contains **30 human messages**. Some are clear, some ambiguous, some suggest illegal workshops, and some request unavailable information. The current program does not yet interpret this inbox.

There are two different dates to keep straight.

For the chat exercises, “today” is **1 April 2026**.

For simulation, orders are replayed from **1 January 2026** onward. The simulator's “day 33” is therefore **3 February 2026**.

This matters because several chat requests mention March deadlines that are already past on April 1. The future assistant must notice that.

7. The supplied simulator

`harness/simulate.py` **is the supplied testing environment.**

Its main components are:

Component	Meaning
`Workshop`	A structured profile for one workshop
`load_workshops()`	Read workshop profiles from CSV
`load_batches()`	Read orders and arrange them by dispatch date
`eligible_workshops()`	Apply the official legal-allocation rules
`Simulator.run()`	Replay orders using a particular allocation policy
`Simulator.report()`	Summarise the resulting performance

For every batch, the simulator:

1. Calculates each workshop's current remaining queue.
2. Gives the batch, workshop profiles, and queues to our allocator.
3. Receives a workshop ID.
4. Checks that the workshop is eligible.
5. Simulates processing, possible rework, transport, and any closure delay.
6. Updates that workshop's queue.
7. Moves to the next batch.

If our allocator chooses an illegal workshop, the simulator raises an error. It does not merely subtract marks from a score.

The normal processing formula is:

```
Processing days = Number of pieces / Pieces produced per day
```

A simulated defective batch takes **50% more processing time**.

The defect probability is a chance that the whole batch requires rework. It is not a prediction that exactly that percentage of individual garments will be defective.

The simulator also provides three simple policies:

- `random`: choose a random eligible workshop.
- `greedy_biggest`: choose the eligible workshop with the highest daily capacity.
- `cheapest`: choose the eligible workshop with the lowest price.

Running `python harness/simulate.py` runs those three policies. **It does not automatically run our Allocator.**

The supplied simulator remains unchanged.

8. The tools and a worked example

`src/tools.py` **contains the checks and calculations our decision-maker uses.**

A *function* is a reusable recipe. You provide inputs; it returns an answer.

The six functions are *pure*: they calculate answers without changing the supplied batch or queues, writing files, contacting an AI model, or remembering previous calls.

Function	Question it answers
`check_eligibility()`	Which workshops are legally allowed to take this batch?
`queue_check()`	How busy is this workshop now, and how fast does it work?
`estimate_eta()`	When would this batch normally finish?
`estimate_cost()`	What would this workshop charge under the simulator's model?
`overload_probe()`	How much work would this allocation add to the queue?
`candidates()`	Show all legal, non-excluded choices with their estimates.

`check_eligibility()` checks three rules in order:

1. Is the workshop active?
2. Can it make this category?
3. Is the quantity within any batch cap?

It returns a reason such as:

```
ok
suspended
cannot make TOPS
batch of 340 exceeds 300-piece cap
```

`queue_check()` reads the current queue supplied by the simulator. It does not keep reusing the initial queue in the CSV.

`estimate_eta()` calculates:

```
Processing = pieces / capacity
Total time = queue + processing + transport lead
ETA = today + rounded total time
Days to spare = due date - ETA
```

Positive spare days mean early. Zero means on the date. Negative means late.

`estimate_cost()` calculates:

```
Total cost = pieces * cost per piece
```

`overload_probe()` calculates:

```
Queue after allocation = current queue + processing time
```

It does **not** add transport to the production queue. Once production finishes, transport can happen while the workshop starts another batch.

`candidates()` combines the results, removes excluded workshops, and sorts by total duration. Exclusions can be full workshop names or IDs.

It does not understand a sentence such as “keep it away from the place that disappointed us last week.” A future language layer must turn that instruction into a verified workshop ID first.

A calculation you can explain by hand

Suppose:

- The batch is 340 TOPS.
- Today is April 1.
- The due date is April 10.
- SteadyHands has a 2.5-day queue.
- Its capacity is 170 pieces/day.
- Its transport lead is two days.
- Its price is 1.30 per piece.

Then:

```
Processing = 340 / 170 = 2 days
Total time = 2.5 + 2 + 2 = 6.5 days
Cost = 340 * 1.30 = 442
Queue after allocation = 2.5 + 2 = 4.5 days
```

Python rounds an exact 6.5 to 6, so this tool estimates April 7, leaving three days to spare.

For the same example, using all initial workshop queues:

Workshop	Estimated total days	ETA	Cost
W6 Nimble Needle	4.62	April 6	408
W1 QuickStitch	6.13	April 7	510
W2 SteadyHands	6.50	April 7	442
W5 GiantWeave	7.85	April 9	374
W3 BudgetWorks	9.48	April 10	272

W4 cannot make TOPS, W7 is suspended, and W8 cannot take 340 pieces.

The speed preference selects W6. A defect preference selects W2. A cost-only preference selects W3.

The estimates describe normal conditions. Unexpected rework or a hidden closure can make completion later. There is also a documented half-day rounding edge case: the tools round duration from today, while the simulator rounds completion from the run's starting date. Those can differ by one day in certain exact-half cases.

Consequently, we should say the tools reproduce the base processing and cost rules, while openly documenting the limits of exact date matching.

9. Objectives: what does “best” mean?

`src/objectives.py` **defines what “best workshop” means.**

An *objective* is a preference. It answers:

> *Among the legal options, what should we prioritise?*

`ObjectiveConfig` stores five weights:

- Lateness.
- Defects.
- Fairness.
- Robustness.
- Cost.

A zero weight ignores a factor. A larger weight makes that factor more influential.

The ready-made presets are:

Preset	Preference
`lateness`	Lateness weight 1
`defect`	Defect weight 1
`fairness`	Fairness weight 1
`robustness`	Robustness weight 1
`hybrid_lateness_fairness`	Lateness 0.7, fairness 0.3
`hybrid_lateness_defect`	Lateness 0.6, defect 0.4

Cost is supported through a custom configuration, although there is no named cost preset.

`load_config()` can read settings from JSON or YAML. This lets someone inspect or change preferences without rewriting the scoring function. Invalid weights and unknown configuration fields are rejected.

`score_terms()` calculates the individual penalties. `score_candidate()` combines them.

A lower score is better. This is a penalty formula, not an AI confidence score.

The actual formula is:

```
late = max(0, -days_to_spare)
t = total_days
share = this workshop's previously allocated pieces / all allocated pieces
flag = 1 if currently suspect, otherwise 0

L = late / (late + 30) + 0.01 * t / (t + 30)
D = expected_defect_rate
F = share
R = (flag + 0.25 * share) / 1.25
C = cost_per_piece / highest workshop cost_per_piece

score = w_lateness * L
      + w_defect * D
      + w_fairness * F
      + w_robustness * R
      + w_cost * C
```

Why divide things? Because days, prices, and probabilities use different scales. Without adjustment, a number such as 30 days might dominate 0.03 defect probability simply because it is numerically larger.

In everyday terms:

- **Lateness:** penalise missed deadlines, with a small preference for faster completion.
- **Defect:** prefer lower rework probability.
- **Fairness:** discourage repeatedly sending most pieces to the same workshop.
- **Robustness:** discourage flagged workshops and excessive concentration.
- **Cost:** prefer lower unit prices.

Fairness here means distributing business volume across workshops. It does not mean equal profit or equal workloads relative to their capacities.

The weights and constants are design choices. They have not been automatically learned or proven optimal. A 0.7 lateness weight does not mean “70% of orders will be on time.”

10. The allocator and decision record

`src/allocator.py` **connects the calculations, preferences, and memory.**

A *class* is a definition for an object that can hold data and perform actions. One `Allocator` object is like one dispatcher with its own notebook.

It remembers:

- The selected objective configuration.
- Pieces allocated to each workshop.
- Expected queues and suspect flags.
- The first decision date.
- The most recent decision record.

This is ordinary stored program state. It is not a model learning from experience.

`__init__()` prepares that notebook.

`__call__()` lets the simulator use the allocator like a function:

```
allocator(batch, workshops, queues)
```

It returns a workshop ID.

`decide()` performs the detailed decision:

1. Observe queues for surprises.
2. Build eligible, non-excluded candidates.
3. Calculate scores.
4. Apply date constraints and awkward-case rules.
5. Choose a workshop.
6. Create an explanation record.
7. Update piece counts and expected queue.

If `must_make_date=True`, it selects among on-time candidates when any exist.

If that setting is false, an objective such as defect minimisation may choose a late workshop even when another could meet the date.

If **all** candidates are late, the allocator marks `none_makes_date`, selects the least-late choice, and offers options to accept lateness or consider splitting.

If no candidates remain, it marks `no_eligible`, selects nothing, and records escalation as an option.

Splitting is not implemented. The record currently suggests a later split-feasibility check.

The other major class, `DecisionRecord`, is the receipt explaining a decision. It contains:

- Order and chosen workshop.
- Objective name.
- Candidate alternatives with estimates and scores.
- Ineligible workshops with reasons.
- Explanation text.
- Constraints.
- Awkward-case status and options.
- Shock evidence.
- Timestamp.

`to_dict()` turns that form into ordinary dictionaries and lists, which can be converted to JSON.

JSON is a structured text format for saving or exchanging records.

There are two practical limits to remember:

- JSON-compatible records are **not yet automatically saved as a complete permanent decision history**.
- A successful `decide()` call updates memory. Calling it twice for the same batch counts that batch twice across the calls.

Before building a screen that repeatedly previews choices, we should separate “show me options” from “confirm this allocation.”

11. Detecting unexpected queue growth

The ShockMonitor notices when a workshop's queue does not shrink as expected.

It is in `src/tools.py`, alongside the six functions, but it is deliberately stateful.

Suppose we predict five days of work remaining today. Tomorrow, without another allocation there, we expect four days.

If the observed queue is twelve days:

```
Unexpected jump = 12 - 4 = 8 days
```

Something has delayed the workshop.

The monitor:

- Stores the queue predicted after our allocation.

- Subtracts elapsed days when checking again.
- Flags a workshop when observed workload exceeds expectation by **more than three days**.
- Records the evidence.
- Clears the flag when the queue comes back within that tolerance.

The methods mean:

Method	Meaning
`observe()`	Compare predictions with current observations
`record_allocation()`	Remember the forecast after sending new work
`is_flagged()`	Check whether the workshop is currently suspect
`clear()`	Remove a recovered workshop's flag

It cannot diagnose a workshop to which it has never sent work because it has no allocation-based expectation for it.

It also cannot prove the cause is closure. Rework can create unexpected queue growth too.

The monitor is not secretly told which workshop closes. Only the evaluator knows that information for checking the result.

The saved default shock experiment shows:

Observation	Result
Hidden target	W2 SteadyHands
Detection	Day 33, February 3
Expected queue	5.38 days
Observed queue	13.94 days
Unexpected increase	8.56 days
Our dispatches while W2 remained flagged	Zero pieces
Recovery	Day 44, February 14

We did send 500 pieces earlier on detection day, **before** the later observation exposed the problem. The monitor cannot react to information it has not yet seen.

During days 34-43, random dispatch sent W2 another 550 pieces, while our default robustness run sent none.

W2 subsequently recovered and received work again.

That demonstrates detection and recovery in this experiment. It does not prove that every objective always avoids flagged workshops or that the robustness setting gives the fastest overall schedule.

12. The heuristic and evaluation files

`src/heuristic.py` **is deliberately simple, and that makes it an important test.**

Its `earliest_finish()` function chooses the eligible workshop with the smallest:

```
Queue + pieces/capacity + transport lead
```

It has no objective configuration, explanation record, or allocation memory.

This is the strong short baseline against which we should judge whether our extra complexity improves scheduling.

`eval/shock_trace.py` performs a different comparison: random dispatch versus our allocator during a closure.

It produces:

- `shock_trace.csv`: daily quantities sent to the affected workshop.
- `shock_events.json`: the evidence for detection and recovery.

The script defaults to the robustness preset and seed 5105. It already enables shock; it does not need an additional `--shock` argument.

13. Tests and measured results

The tests check programming behaviour, while evaluation measures allocation performance.

Those are related but different.

Tests ask questions such as:

- Does our eligibility check match the simulator?
- Is OldMill always rejected?
- Is a 301-piece batch rejected by FreshStart?
- Is the hand-calculated ETA correct?
- Can a decision record be converted to JSON?
- Do flags clear after recovery?
- Does the allocator avoid illegal selections throughout a simulation?

The current suite passed **89 test cases**:

Test file	Cases
`test_tools.py`	38
`test_allocator.py`	37
`test_shock_monitor.py`	14

`tests/conftest.py` helps Python find our source modules during these tests.

Passing 89 tests does not mean 89% accuracy, 89 successful AI conversations, or proof that no bugs remain.

The fresh performance comparison gives us encouraging results and an important limitation.

These figures use the supplied 120 orders, seed 5105, and normal conditions:

Policy	Average turnaround	Late batches	Total cost	Largest workshop's piece share
Random	21.52 days	41/120	111,780	22.62%
Biggest workshop	49.02 days	101/120	108,430	70.16%
Cheapest workshop	105.13 days	112/120	85,960	70.16%
Earliest-finish heuristic	8.05 days	2/120	115,640	25.56%
Our lateness allocator	8.05 days	2/120	115,640	25.56%
Our defect allocator	162.51 days	114/120	129,920	70.16%
Our fairness allocator	19.20 days	28/120	116,045	17.49%
Lateness/fairness hybrid	12.20 days	6/120	113,385	21.02%

What should you conclude?

- Our lateness setting substantially improves turnaround and lateness compared with the three supplied baselines.
- **It exactly matches the earliest-finish heuristic in both tested normal and shock runs. It does not beat that heuristic here.**
- Cheapest wins on spending.
- Fairness distributes pieces more evenly, with a speed trade-off.
- Defect minimisation can concentrate work at limited high-quality capacity and create severe queues.

During shock, the robustness setting averages **18.61 days**, while earliest finish averages **8.38 days**. Therefore, the successful shock-detection example should not be presented as proof that robustness wins overall.

These are development measurements. The final report needs repeatable evaluation, additional scenarios, disclosed development use, held-out evidence, and individual failure analysis.

The companion learning guide contains the full normal and shock tables for every preset.

14. What to build next and the deadlines

The next work should complete the dispatcher workflow and its evidence.

A sensible order is:

1. **Choose the team's primary objective.** Explain why that business preference matters. Keep the required additional objective and hybrid comparisons.
2. **Define expected answers for the 30 messages.** Identify what to extract, what needs clarification, which workshop suggestions must be refused, and which facts cannot be answered.

3. **Build validated message intake.** Resolve order ID, quantity, category, dates, exclusions, and constraints. Preserve who imposed a constraint.
4. **Add the language model.** Use it to interpret requests, ask questions, and explain tool results. Keep arithmetic in Python.
5. **Separate preview from confirmation and save full records.** Prevent repeated previews from changing allocation history. Preserve original messages, exact settings, input evidence, and actual timestamps.
6. **Build the Streamlit interface.** Show alternatives, estimates, restrictions, recommendation, and explanations clearly.
7. **Decide the scope of splitting.** Either implement a genuine feasibility calculation or clearly retain it as an unimplemented suggestion.
8. **Complete evaluation and presentation artifacts.** Compare all required policies, analyse at least ten individual failures, evaluate all 30 messages, and score decision correctness separately from explanation faithfulness.

According to the supplied v4 specification:

Milestone	Date
Sprint 1	16 October 2026
Sprint 2	30 October 2026
Sprint 3	13 November 2026
Final ZIP	15 November 2026, 11:59 PM

Sprint 1 is a five-minute live presentation plus questions. Later sprints use five-minute presentation videos plus live questions.

The final submission includes setup instructions, slides, a demo video, documentation, evaluation, contributions, and evidence-based strategic recommendations. The learning guide lists the exact filenames and submission details. Source: `Documents_Capstone/Capstone_Project_Spec_v4.pdf`.

The sprint-example PDF shows how to explain architecture, progress, risks, and a roadmap. Its example is an email assistant; its features and results should not be presented as achievements of our dispatch project. Source: `Documents_Capstone/DSS5105_Sprint_Review_Example.pdf`.

15. A presentation explanation to practise

For your presentation, this is an accurate explanation you can practise saying:

> *Our project helps a garment-factory dispatcher choose an outside workshop for each batch. We check capability, suspension status, and batch limits, then calculate waiting time, production time, transport, cost, and defect risk. A configurable scoring rule ranks the legal choices, and a decision record explains the result.*
>
> *We also monitor unexpected queue growth to detect possible disruptions. Our current implementation is the deterministic decision engine; the conversational AI and user interface are still being developed.*
>
> *In our current experiments, the lateness setting beats the three supplied baselines on turnaround and lateness, but matches the simple earliest-finish heuristic. We will report that honestly and evaluate the*

> *additional value of our constraints, explanations, and conversational workflow.*

To study the code, start with the 340-piece calculation, then follow `tools.py`, `objectives.py`, `allocator.py`, and ShockMonitor. Use the full learning guide alongside those files; it explains the Python syntax and each function in more depth.

Only the learning-guide document was added during the original project review. The application and simulator were not modified. Preparing these PDFs likewise does not change the application behaviour.

16. Source documents and scope

- `Track2_dispatch/docs/project_learning_guide.md`: complete companion learning guide.
- `Track2_dispatch/pdfs/Factory_Primer.pdf`: manufacturing setting.
- `Documents_Capstone/Capstone_Project_Tracks_v3.pdf`: latest supplied Track 2 requirements.
- `Documents_Capstone/Capstone_Project_Spec_v4.pdf`: latest supplied capstone specification, evaluation and deadlines.
- `Documents_Capstone/DSS5105_Sprint_Review_Example.pdf`: illustrative presentation structure.
- `Track2_dispatch/src/`, `harness/`, `tests/`, `data/`, and `eval/`: implementation and evidence reviewed on 14 September 2026.

The other capstone specification and track versions were compared during the review. This document explains the reviewed implementation; it does not claim that unfinished features exist or that later course announcements have been checked.